

Cruscotto Degenze

Prompt di sviluppo — testo da copiare e incollare

Redatto da Mihnea Sever Molnar · Versione 2.0 · 14 settembre 2026

Istruzioni d'uso

I due riquadri contengono il testo esatto da incollare nella conversazione. Non vanno riassunti né riscritti: le parti che sembrano ripetitive, come l'elenco dei campi delle tabelle o la sezione "cosa non fare", sono quelle che evitano che venga proposta un'architettura sovradimensionata o un modello dati diverso da quello progettato.

- Il **Prompt A** serve a costruire l'applicazione in locale sul PC. Va usato finché il programma non funziona e non è stato provato dal personale.
- Il **Prompt B** serve a pubblicare l'applicazione sulla macchina virtuale. Va usato solo dopo, in una conversazione nuova.
- Ogni prompt va incollato per intero come primo messaggio della conversazione. Poi si procede a domande e risposte.
- Se durante lo sviluppo cambia una decisione, va aggiornato il testo del prompt prima di ripartire in una nuova conversazione.

PROMPT A — sviluppo in locale

[copiare da qui](#)

Ciao. Devo realizzare un'applicazione web e voglio che tu mi guidi passo per passo. Leggi tutta la specifica prima di iniziare a scrivere codice.

CHI SONO

Sono un amministratore IT junior in una casa di cura privata italiana. Conosco Python a livello di scripting e analisi dati, ma non ho mai sviluppato un'applicazione web strutturata: non conosco Flask, non conosco SQLAlchemy e non so come si organizza un progetto web. Quando usi un concetto o una convenzione che dai per scontata, spiegamela. Lavoro su Windows 11 con Visual Studio Code.

COSA DEVO REALIZZARE

Un'applicazione web interna chiamata "Cruscotto Degenze": l'agenda dei pazienti ricoverati, usata da medici e infermieri per annotare giorno per giorno l'andamento della degenza. Sostituisce una gestione fatta oggi a voce e su carta.

STACK TECNOLOGICO OBBLIGATORIO

- Python 3.12, Flask, template Jinja2.
- SQLAlchemy come ORM. Questo è vincolante: in una fase successiva devo passare a PostgreSQL e voglio farlo cambiando solo la stringa di connessione.
- SQLite in questa fase, database in un unico file degenze.db.
- Flask-Login per la sessione.
- HTML, CSS e JavaScript vanilla lato client. Nessun framework frontend.
- Nessuna dipendenza che richieda compilatori, servizi esterni o account cloud.

STRUTTURA DEL PROGETTO

```
C:\cruscotto\  
  .venv\  
  app.py          avvio dell'applicazione e rotte  
  modelli.py      tabelle SQLAlchemy  
  permessi.py     matrice dei ruoli e decoratori di controllo
```

inizializza_db.py	creazione del database e dati iniziali
degenze.db	il database
templates\	base.html, login.html, agenda.html, utenti.html
static\	stile.css, agenda.js

MODELLO DATI

Usa esattamente queste tabelle, con questi nomi di campo.

utenti

id	INTEGER	chiave primaria
username	TEXT	univoco
password_hash	TEXT	
nome_completo	TEXT	
ruolo	TEXT	ADMIN MEDICO INFERMIERE OSS AMMINISTRAZIONE
attivo	BOOLEAN	predefinito vero
creato_il	TIMESTAMP	

reparti

id	INTEGER	chiave primaria
codice	TEXT	univoco: AOUN S1 S2
nome	TEXT	"AOUN", "Setting 1 - Low Care 1", "Setting 2 - Low Care 2"
giorni_max	INTEGER	10 per AOUN, 20 per S1 e S2

pazienti

id	INTEGER	chiave primaria
reparto_id	INTEGER	chiave esterna verso reparti
nome	TEXT	
cognome	TEXT	
letto	TEXT	
gravita	TEXT	nv verde giallo rosso (nv = non valutato)
data_inserimento	DATE	
stato	TEXT	RICOVERATO
creato_il	TIMESTAMP	

note_giornaliere

id	INTEGER	chiave primaria
paziente_id	INTEGER	chiave esterna verso pazienti, cancellazione a cascata
data	DATE	
testo	TEXT	
autore_id	INTEGER	chiave esterna verso utenti
aggiornato_il	TIMESTAMP	
vincolo di unicita' sulla coppia (paziente_id, data)		

note_cliniche

id	INTEGER	chiave primaria
paziente_id	INTEGER	chiave esterna verso pazienti, cancellazione a cascata
tipo	TEXT	MEDICO INFERMIERE
testo	TEXT	
autore_id	INTEGER	chiave esterna verso utenti
creato_il	TIMESTAMP	

audit_log

id	INTEGER	chiave primaria
utente_id	INTEGER	chiave esterna verso utenti
azione	TEXT	es. CREA_PAZIENTE, MODIFICA_NOTA, ELIMINA_PAZIENTE
entita	TEXT	nome della tabella interessata
entita_id	INTEGER	
quando	TIMESTAMP	

REGOLA CENTRALE: LA GRIGLIA DEI GIORNI

Le note giornaliere sono righe, non colonne. Non creare campi giorno1, giorno2 e cosi' via: la griglia si ricostruisce a video.

- Il giorno 1 di un paziente e' la sua data_inserimento; il giorno N e' data_inserimento

- piu' N-1 giorni; si arriva fino a giorni_max del suo reparto.
- Pazienti diversi hanno date di inserimento diverse, quindi l'intestazione delle colonne dice "Giorno 1 ... Giorno N" e la data effettiva compare dentro ogni cella, in piccolo sopra il testo della nota. Non mettere una riga di date condivisa fra tutti i pazienti.
- Per le date di calendario usa datetime.date in Python e il tipo Date in SQLAlchemy, mai datetime con fuso orario: aritmetica con timedelta su oggetti date, altrimenti si producono errori di un giorno al cambio dell'ora legale.
- Per creato_il e aggiornato_il usa invece un timestamp.

RUOLI E PERMESSI

- ADMIN: lettura e scrittura su tutto, gestione utenti, eliminazione pazienti.
- MEDICO: legge tutto, scrive le note giornaliere, scrive le sole note cliniche di tipo MEDICO. Vede le note infermieristiche ma non puo' modificarle.
- INFERMIERE: legge tutto, scrive le note giornaliere, scrive le sole note cliniche di tipo INFERMIERE. Vede le note mediche ma non puo' modificarle.
- OSS: sola lettura.
- AMMINISTRAZIONE: sola lettura piu' esportazione CSV.

Il controllo dei permessi va fatto lato server su ogni rotta, con un decoratore.

Nascondere un pulsante nel template non e' una misura di sicurezza.

SCHERMATE

LOGIN. Form con nome utente e password. Nessuna registrazione autonoma. Messaggio di errore generico, senza distinguere fra utente inesistente e password sbagliata.

AGENDA PAZIENTI, pagina iniziale dopo il login. Menu di navigazione a sinistra con le voci Agenda pazienti, Gestione utenti (visibile solo ad ADMIN) ed Esci. Al centro tre sezioni impilate, una per reparto, nell'ordine AOUP, Setting 1, Setting 2. Ogni sezione contiene:

- intestazione con nome del reparto, numero di pazienti e giorni di degenza massimi;
- una tabella a due parti: a sinistra le colonne fisse cognome, nome, letto, gravita' e azioni; a destra la griglia dei giorni, che scorre orizzontalmente;
- la gravita' e' un indicatore colorato che cambia a click ciclando nv, verde, giallo, rosso;
- per ogni paziente i pulsanti "Note medico", "Note infermiere" ed "Elimina";
- un pulsante "Inserisci paziente" nell'intestazione del reparto;
- un pulsante "Scarica CSV" visibile solo ad ADMIN e AMMINISTRAZIONE;
- la colonna della data odierna evidenziata e l'ultimo giorno utile segnalato.

GESTIONE UTENTI, solo ADMIN. Elenco degli utenti, creazione di un nuovo utente con nome completo, username, password e ruolo, disattivazione di un utente esistente. Un utente non puo' disattivare se' stesso.

COMPORTAMENTI

- La cella del giorno si salva quando l'utente esce dal campo. Se viene svuotata, la riga corrispondente in note_giornaliere va eliminata.
- Le note cliniche sono voci in sola aggiunta: una nota nuova non sostituisce le precedenti, e ognuna mostra autore e data.
- L'eliminazione di un paziente chiede conferma e cancella a cascata le sue note.
- Ogni scrittura registra l'autore e produce una riga in audit_log.
- L'esportazione CSV usa il punto e virgola come separatore e il BOM UTF-8, altrimenti Excel italiano sbaglia le colonne e le lettere accentate.

PASSWORD: TENIAMOLO SEMPLICE

Le utenze le crea l'amministratore e le password le comunica di persona. Non implementare registrazione autonoma, recupero password, scadenze, requisiti di complessita' o secondo fattore. L'unica cosa da fare e' salvare l'hash con generate_password_hash di Werkzeug e verificare con check_password_hash.

COME VOGLIO CHE LAVORIAMO

1. Come prima risposta dammi soltanto: l'elenco dei pacchetti da installare, il contenuto del file requirements.txt e l'ordine in cui costruiremo i file. Niente codice ancora.
2. Poi procediamo un file alla volta. Per ogni file: il codice completo, dove salvarlo, una spiegazione in italiano di cosa fa e come verificare che funzioni. Poi fermati e aspetta la mia conferma prima di passare al file successivo.
3. Commenti nel codice in italiano.

4. Se un punto della specifica ti sembra ambiguo o in contraddizione, chiedimelo invece di decidere da solo.
5. Quando ti segnalo un errore, dimmi la causa prima della correzione.

COSA NON FARE

- Niente Docker, niente React o altri framework frontend, niente API REST separata.
- Niente Alembic o migrazioni in questa fase.
- Niente integrazione con Active Directory o LDAP per ora.
- Niente blueprint Flask o struttura a package: in questa fase voglio pochi file piatti.
- Non aggiungere funzionalita' che non ho chiesto. Se pensi che ne serva una, proponimela e aspetta la mia risposta prima di scriverla.

Inizia rispondendo al punto 1.

Fine del Prompt A.

Ciao. Ho un'applicazione web Flask funzionante che gira in locale sul mio PC e devo pubblicarla su una macchina virtuale aziendale. Guidami passo per passo con i comandi esatti.

COSA HO GIA'

Un'applicazione chiamata "Cruscotto Degenze", scritta in Python 3.12 con Flask, Jinja2, SQLAlchemy e Flask-Login, con database SQLite in un file degenze.db. E' l'agenda dei pazienti ricoverati di una casa di cura: sei tabelle (utenti, reparti, pazienti, note_giornaliere, note_cliniche, audit_log), cinque ruoli utente, pagine renderizzate lato server. Finora e' sempre partita con flask run in ascolto su 127.0.0.1.

AMBIENTE DI DESTINAZIONE

- Macchina virtuale Ubuntu Server 24.04 LTS nel datacenter aziendale, indirizzo IP statico, nessuna esposizione verso Internet.
- Rete locale della struttura sanitaria, con dominio Active Directory interno e DNS interno.
- Client: PC Windows con Chrome ed Edge.
- Circa venti utenti registrati, non piu' di dieci collegati contemporaneamente.
- Amministro la VM via SSH e ho i privilegi di sudo.

COSA DEVO OTTENERE

- PostgreSQL sulla stessa VM, con database e utente applicativo dedicati.
- Passaggio da SQLite a PostgreSQL. Dimmi tu se conviene migrare i dati o ricreare lo schema da zero reinserendo solo utenti e reparti, e perche'.
- Configurazione letta da variabili d'ambiente, non scritta nel codice sorgente.
- Unicorn che esegue l'applicazione in ascolto solo su 127.0.0.1:8000.
- Nginx come reverse proxy sulla porta 80 verso Unicorn.
- Un servizio systemd che avvia Unicorn al boot e lo riavvia in caso di crash.
- Accesso consentito solo dalle sottoreti della LAN, tutto il resto bloccato.
- Backup automatico giornaliero del database, conservazione trenta giorni, su un percorso diverso da quello del database.
- Log applicativi su file con rotazione.

COME VOGLIO CHE LAVORIAMO

1. Come prima risposta dammi soltanto la sequenza dei passi, numerata, senza comandi. Così' verifico che l'ordine sia giusto prima di eseguire qualcosa.
2. Poi affrontiamo un passo alla volta: comandi esatti nell'ordine di esecuzione, cosa fa ciascuno, e come verificare che quel passo abbia funzionato prima di andare avanti.
3. I file di configurazione (unit systemd, sito nginx, file delle variabili d'ambiente) dammeli completi, non a frammenti.
4. Indicami esplicitamente le modifiche da fare al codice dell'applicazione, file per file.
5. Se un comando puo' interrompere il servizio o perdere dati, dimmelo prima.

COSA NON FARE

- Niente Docker, niente Kubernetes, niente Ansible.
- Niente Let's Encrypt: la macchina non e' esposta su Internet. Se serve HTTPS spiegami come emettere un certificato dalla CA interna del dominio, ma come passo separato e successivo.
- Niente bilanciatori, cache, code di messaggi o servizi aggiuntivi.
- Non riscrivere l'applicazione: solo le modifiche minime necessarie alla messa in produzione.

Inizia rispondendo al punto 1.

Fine del Prompt B.

Prima di usare il Prompt B

L'applicazione tratta dati relativi alla salute di persone identificabili. Prima di metterla in rete, anche solo sulla LAN interna, va coinvolto il Responsabile della protezione dei dati della struttura per la valutazione del trattamento e l'inserimento nel registro. Fino a quel momento, prototipo e prove vanno fatti esclusivamente con nomi di fantasia.